DOSSIER PROFESSIONNEL

DÉVELOPPEUR WEB ET WEB MOBILE

RELAY

Application de gestion de projets et de tâches collaboratives



Présenté par : Yvo Toso

Centre de formation : Institut de la PME

Session d’examen : Octobre 2026

Sommaire

1. Introduction 3
1.1 Présentation du projet 3
1.2 Objectifs du projet 4
1.3 Fonctionnalité complémentaire : traitement des réunions et IA 4
2. Expression des besoins 6
2.1 Problématique 6
2.2 Limites du projet 6
3. Analyse et conception 8
3.1 Analyse fonctionnelle 8
3.2 Diagramme de classes 9
3.3 Maquettes 11
4. Architecture technique 14
4.1 Vue d'ensemble 14
4.2 Environnement de développement 14
4.3 Technologies utilisées 15
4.3.1 Application principale 15
4.3.2 Enregistrement des réunions 18
4.3.3 Transcription audio 18
4.3.4 Intelligence Artificielle locale et traitement des transcriptions 19
5. Développement Back-end 21
5.1 Base de donnée 21
5.2 Contrôleurs 22
5.3 Services 24
5.4 API 25
6. Développement Front-end 28
6.1 Structure des interfaces 28
6.2 Intégration graphique 29
6.3 JavaScript 31
6.3.1 ajout asynchrone d'un collaborateur 32
7. Déploiement 35
7.1. Choix de l'hébergement 35
7.2. Configuration du nom de domaine et du réseau 35
7.3. Déploiement avec Docker 36
8. Difficultés et améliorations 37
8.1 Difficultés rencontrées 37
8.1.1 Gestion des notifications en temps réel avec Mercure 37
8.1.2 Récupération des enregistrements depuis Craig 38
8.1.3 Utilisation de la carte graphique par Docker 38
8.2 Améliorations futures 39

9. Conclusion..... 41
Annexes..... 1
Diagramme de classes complet..... 1
Concepteur BDD..... 2
Wireframes..... 2
Maquettes..... 4
Task DTO..... 5
Captures d’écrans..... 6

1. Introduction

1.1 Présentation du projet

Au début de ma formation, je savais que je préparerais le titre professionnel **Développeur Web et Web Mobile (DWWM)**, avec pour objectif de poursuivre ensuite vers le titre **Concepteur Développeur d'Applications (CDA)**. J'ai donc souhaité choisir un projet qui puisse répondre aux attentes du DWWM tout en disposant d'un potentiel d'évolution suffisant pour être approfondi dans le cadre du CDA.

Je souhaitais également travailler sur un projet qui me motive personnellement et qui me permette d'explorer de nouvelles technologies. L'un des domaines qui m'intéresse particulièrement est celui de l'intelligence artificielle, notamment la possibilité de faire fonctionner des modèles relativement légers en local et de les intégrer à une application concrète.

C'est dans ce contexte que j'ai imaginé une application de gestion de projet destinée au travail en équipe. L'objectif principal est de permettre à une équipe de créer et suivre ses projets et ses tâches, de les attribuer à différents collaborateurs et d'en suivre l'avancement.

J'ai souhaité aller plus loin qu'un simple outil de gestion de tâches en y intégrant un système permettant de conserver les enregistrements des réunions et d'en générer automatiquement une transcription. Cette transcription peut ensuite être exploitée afin de faciliter l'identification des actions évoquées au cours d'une réunion et, à terme, la création ou la mise à jour des tâches correspondantes.

Le projet repose ainsi sur deux fonctionnalités complémentaires : **la gestion de projet** et **l'exploitation de réunions grâce à l'intelligence artificielle**. L'objectif est de rapprocher les échanges ayant lieu pendant les réunions du suivi opérationnel du projet, afin de limiter la perte d'informations et de faciliter le suivi des actions décidées par l'équipe.

Dans sa première version, le projet est volontairement limité à un périmètre permettant de démontrer les principales compétences nécessaires au titre DWWM. Donc la fonctionnalité gestion de projet. Il a cependant été conçu dès le départ avec une architecture et des fonctionnalités pouvant être enrichies par la suite. L'objectif est notamment de pouvoir approfondir le projet dans le cadre de ma poursuite en CDA, en développant les fonctionnalités liées à l'intelligence artificielle et l'automatisation de la gestion des tâches.

1.2 Objectifs du projet

L'objectif principal de l'application est de fournir à une équipe un outil permettant de centraliser la gestion d'un projet et de ses tâches.

Objectifs fonctionnels :

- créer et gérer des projets ;
- créer, modifier et supprimer des tâches ;
- attribuer des tâches aux différents collaborateurs ;
- suivre l'état et l'avancement des tâches ;
- organiser les tâches au sein d'un projet ;
- permettre aux collaborateurs de consulter et de mettre à jour les tâches qui leur sont attribuées.

Objectifs techniques :

- développer une application web complète avec Symfony ;
- mettre en œuvre une architecture MVC ;
- concevoir des interfaces web avec Twig, HTML et CSS ;
- rendre les interfaces interactives grâce à JavaScript ;
- utiliser une base de données relationnelle pour stocker les données de l'application ;
- mettre en œuvre l'accès aux données et les opérations CRUD ;
- développer la logique métier côté serveur ;
- utiliser des requêtes asynchrones lorsque cela est nécessaire pour certaines interactions de l'interface ;
- utiliser et exposer des endpoints API pour les fonctionnalités qui le nécessitent ;
- conteneuriser les différents services de l'environnement avec Docker.

Cette partie constitue le **cœur du projet présenté dans le cadre du DWWM**. Elle permet de mettre en pratique les différentes compétences nécessaires au développement d'une application web, depuis la conception des interfaces jusqu'à la gestion des données, la logique métier et le déploiement.

1.3 Fonctionnalité complémentaire : traitement des réunions et IA

En complément de la gestion de projet, j'ai souhaité expérimenter l'intégration de fonctionnalités liées à l'intelligence artificielle.

L'objectif est de pouvoir exploiter les enregistrements de réunions afin de générer automatiquement une transcription et, à terme, d'utiliser cette transcription pour faciliter l'identification des actions et des tâches évoquées pendant les échanges.

Objectifs fonctionnels :

- récupérer les enregistrements des réunions ;
- traiter les enregistrements audio ;
- générer automatiquement une transcription ;
- exploiter les transcriptions afin de faciliter l'identification des actions et des tâches.

Objectifs techniques :

- intégrer un traitement utilisant un modèle d'intelligence artificielle exécuté localement ;
- traiter des enregistrements audio multipistes ;
- faire communiquer plusieurs services ;
- automatiser la chaîne de traitement entre la récupération des enregistrements et la génération de la transcription.

Cette partie constitue une **extension du projet**. Elle n'est pas indispensable au fonctionnement principal de l'application de gestion de projet et s'inscrit davantage dans une perspective de poursuite vers le titre **CDA**.

Elle permet néanmoins de montrer les possibilités d'évolution de l'application ainsi que les expérimentations techniques réalisées au cours du développement.

2. Expression des besoins

2.1 Problématique

Dans le cadre du travail en équipe, la gestion d'un projet nécessite de suivre un grand nombre d'informations : les différentes tâches à réaliser, leur état d'avancement, les personnes auxquelles elles sont attribuées ainsi que les évolutions apportées au projet.

Lorsque ces informations sont dispersées entre différents outils ou communiquées principalement lors d'échanges entre les membres de l'équipe, il peut devenir plus difficile de savoir quelles tâches doivent être réalisées, par qui et dans quel état elles se trouvent.

Le besoin identifié est donc de disposer d'un espace centralisé permettant à une équipe de regrouper les informations liées à ses projets et de faciliter le suivi des tâches.

La problématique retenue pour le projet est ainsi la suivante :

> **Comment permettre à une équipe de centraliser la gestion de ses projets et de ses tâches afin de faciliter leur organisation, leur attribution et leur suivi ?**

L'application doit répondre à ce besoin en proposant une interface permettant aux utilisateurs de consulter les projets auxquels ils participent, de gérer les tâches associées et de suivre leur évolution.

2.2 Limites du projet

Afin de conserver un périmètre adapté au développement et à la présentation du titre professionnel **Développeur Web et Web Mobile (DWWM)**, certaines fonctionnalités ne font pas partie du périmètre principal du projet.

Les principales limites retenues sont les suivantes :

- l'application est destinée à la gestion de projets et de tâches et ne cherche pas à couvrir l'ensemble des fonctionnalités d'un outil professionnel de gestion de projet ;
- seuls deux rôles sont actuellement définis au niveau d'un projet : **chef de projet** et **collaborateur** ;
- la gestion des droits reste volontairement limitée à ces rôles et ne propose pas de système de permissions entièrement personnalisable ;

- les fonctionnalités de communication entre les membres de l'équipe sont une fonctionnalité future de l’application ;
- l'application ne propose pas de système complet de planification ou de gestion avancée des ressources ;
- les fonctionnalités liées à l'enregistrement, à la transcription et à l'exploitation des réunions constituent une extension du projet et ne sont pas nécessaires au fonctionnement principal de la gestion de projet ;
- les fonctionnalités d'analyse automatique des transcriptions et de génération de tâches à l'aide de l'intelligence artificielle restent limitées dans cette version.

Ces limites permettent de conserver un périmètre maîtrisable pour la réalisation du projet DWWM, tout en laissant des possibilités d'évolution pour une version ultérieure du projet.

3. Analyse et conception

3.1 Analyse fonctionnelle

L'application repose sur un système de rôles permettant de définir les droits d'un utilisateur en fonction du projet auquel il participe.

Un utilisateur possède un compte général, mais peut avoir un rôle différent pour chacun des projets auxquels il est associé. Deux rôles sont actuellement définis au niveau d'un projet :

- **Chef de projet** ;
- **Collaborateur**.

Utilisateur

Un utilisateur non connecté peut :

- créer un compte
- se connecter

Un utilisateur connecté peut :

- créer un nouveau projet ;
- accéder aux projets auxquels il participe ;
- consulter les informations et les tâches associées à ses projets.

Lorsqu'un utilisateur crée un projet, il en devient automatiquement le **chef de projet**.

Chef de projet

Le chef de projet dispose de droits supplémentaires lui permettant de gérer son projet et son équipe.

Il peut :

- modifier les informations de son projet ;
- ajouter des collaborateurs au projet ;
- créer des tâches ;
- modifier les tâches du projet ;
- gérer l'attribution des tâches ;
- accepter une tâche proposée à partir du contenu d'une réunion.

Le chef de projet a aussi les mêmes possibilités d’action qu’un collaborateur

Collaborateur

Le collaborateur peut participer à la gestion des tâches du projet auquel il est associé.

Il peut :

- ajouter une tâche ;
- modifier une tâche qu'il a créée ;
- modifier une tâche qui lui est attribuée ;
- consulter les tâches associées au projet.
- consulter les réunions enregistrées
- lancer une transcription

Les droits disponibles sont ainsi déterminés par le rôle de l'utilisateur dans le projet. Ce fonctionnement permet notamment à un même utilisateur d'être **chef de projet sur un projet et collaborateur sur un autre**.

3.2 Diagramme de classes

Pour le **Minimum Viable Product (MVP)**, j'ai identifié six entités principales. Ces entités sont représentées sous forme de classes dans l'application et correspondent aux principales tables utilisées pour stocker les données en base.

Le diagramme de classes permet de représenter la structure des données de l'application, d'identifier les informations nécessaires au fonctionnement des différentes fonctionnalités et de définir les relations entre les entités.

Les entités

User : Représente un utilisateur de l'application. Il possède un compte et peut participer à plusieurs projets.

Project : Représente un projet créé dans l'application. Il contient notamment son nom ainsi que les différentes options permettant de définir son fonctionnement.

ProjectUser : Représente l'association entre un utilisateur et un projet. Cette entité permet de stocker les informations propres à la participation d'un utilisateur à un projet, notamment son rôle et ses permissions.

Cette entité permet également de gérer le fait qu'un même utilisateur puisse avoir un rôle différent selon le projet auquel il participe.

Task : Représente une tâche appartenant à un projet. Une tâche peut être associée à un utilisateur du projet et possède plusieurs versions de son contenu afin de conserver son historique.

TaskContent : Représente une version du contenu d'une tâche. Elle contient notamment le titre, la description, le statut, la priorité, la date de début, la date de fin et la date de modification.

Le contenu de la tâche est volontairement séparé de l'entité **Task** afin de permettre de conserver les différentes versions d'une tâche et ainsi de garder un historique de ses modifications.

Meeting : Représente une réunion enregistrée dans le cadre d'un projet. Elle permet notamment de stocker les informations associées à la réunion, telles que son nom, sa date de création, son enregistrement audio et les informations issues de sa transcription.

Les relations entre les entités

Les principales relations entre les entités sont les suivantes :

- **User - Project : ManyToMany**
  Un utilisateur peut participer à plusieurs projets et un projet peut comporter plusieurs utilisateurs. La relation est portée par l'entité **ProjectUser**.
- **Project - Meeting : OneToMany**
  Un projet peut être associé à plusieurs réunions, tandis qu'une réunion appartient à un seul projet.
- **Project - Task : OneToMany**
  Un projet peut contenir plusieurs tâches, tandis qu'une tâche appartient à un seul projet.
- **Task - TaskContent : OneToMany**
  Une tâche peut posséder plusieurs versions de son contenu afin de conserver son historique. Une version de contenu appartient à une seule tâche.
- **Task - TaskContent principal : OneToOne**
  Une tâche possède un contenu actuellement considéré comme son contenu principal. Ce contenu principal correspond à une seule tâche.
- **Meeting - Task : OneToMany**
  Une réunion peut être associée à plusieurs tâches. Une tâche peut être associée à une seule réunion.
- **ProjectUser - Task : OneToMany**
  Un utilisateur associé à un projet peut avoir plusieurs tâches qui lui sont attribuées. Une tâche est attribuée à un seul utilisateur du projet.
- **ProjectUser - TaskContent : OneToMany**
  Un utilisateur associé à un projet peut être associé à plusieurs contenus de tâches, notamment en tant qu'utilisateur ayant créé ou modifié ces contenus. Un contenu de tâche est associé à un utilisateur du projet pour chacune de ces informations.

- **ProjectUser - Meeting : ManyToMany**
  Une réunion peut concerner plusieurs utilisateurs d'un projet et un utilisateur peut participer à plusieurs réunions.

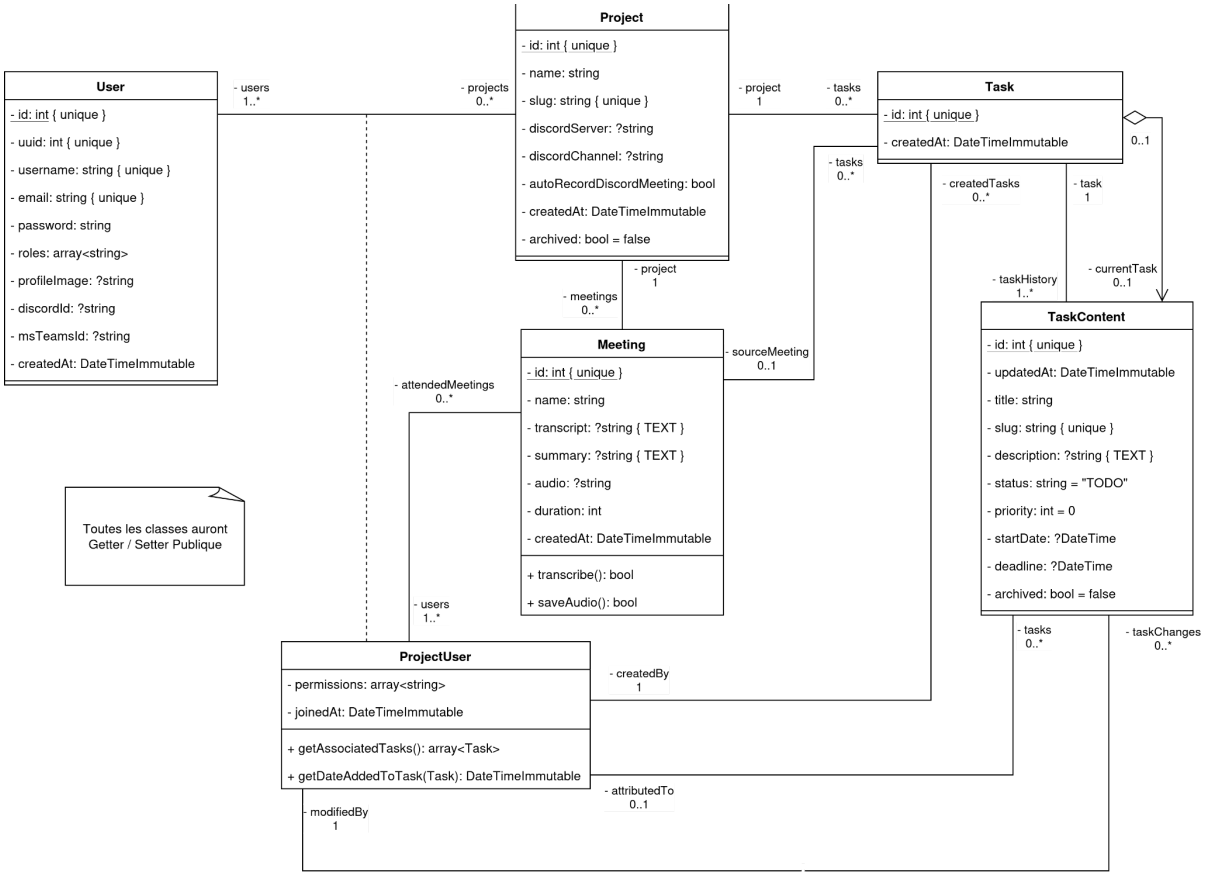


Diagramme de classes pour le DWWM, diagramme pour application finale [en annexe]

3.3 Maquettes

Pour concevoir les maquettes de l'application, j'ai utilisé l'outil en ligne **Figma**.

L'application étant destinée à être utilisée dans un environnement professionnel, j'ai souhaité mettre en place une interface moderne, sobre et neutre, afin de privilégier la lisibilité et de ne pas surcharger visuellement les différentes pages.

J'ai tout d'abord défini une **charte graphique**, comprenant notamment une palette de couleurs et une police d'écriture communes à l'ensemble de l'application. J'ai prévu cette charte pour les deux thèmes de l'application : **mode clair (light mode)** et **mode sombre (dark mode)**.

Pour les couleurs principales, j'ai choisi des teintes noires et blanches proches du bleu afin d'obtenir une interface sobre et confortable à utiliser. Une couleur verte est

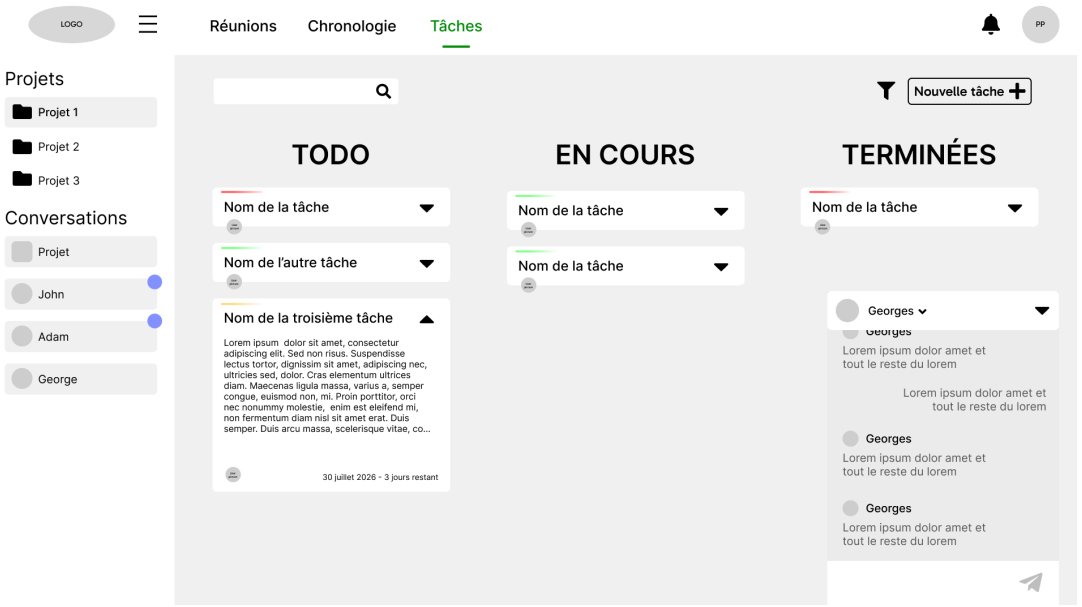
utilisée comme couleur d’accentuation afin de mettre en évidence certains éléments importants et les principales actions de l’utilisateur.





J’ai ensuite réalisé les **wireframes** des différentes pages pour l’interface bureau et mobile. Cette étape m’a permis de réfléchir à l’organisation des informations et au placement des différents éléments de l’interface avant de m’occuper de l'aspect graphique. Par exemple, le wireframe présenté ci-dessous correspond à la page d’un projet.

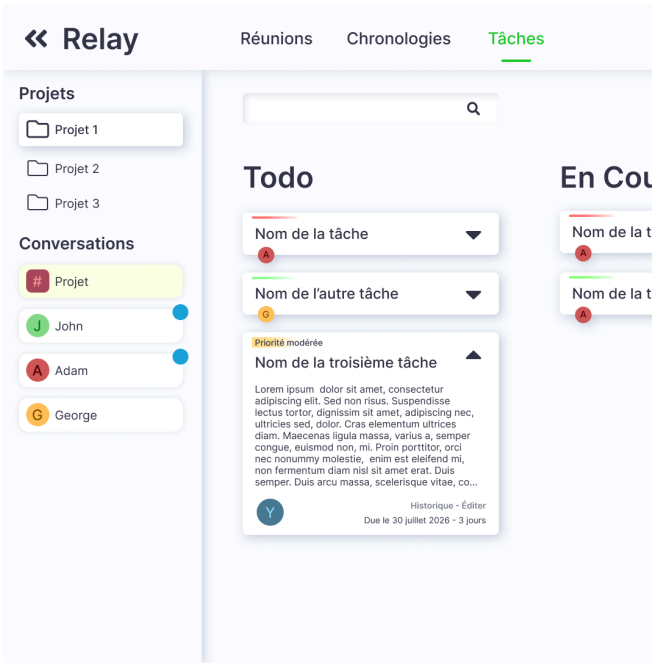
Les autres wireframes sont [disponibles en annexe.]

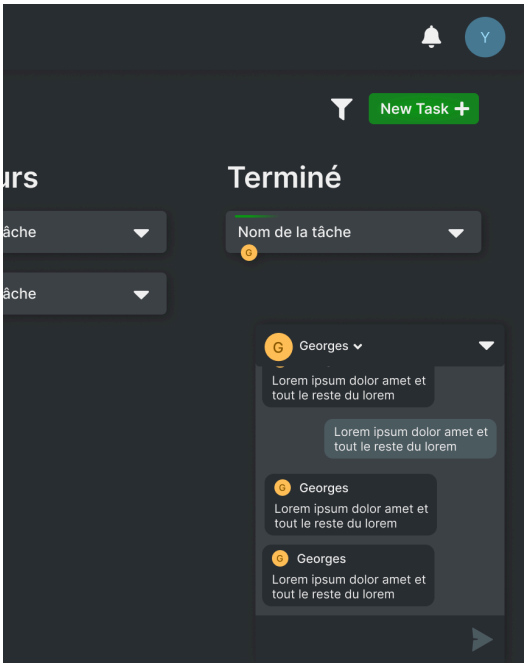


Une fois la structure des pages définie, j'ai réalisé les **maquettes finales** en appliquant la charte graphique précédemment définie. Ces maquettes permettent d'avoir une représentation plus précise du rendu final de l'application et servent également de référence lors du développement des interfaces.

J'ai choisi un design volontairement simple, basé notamment sur des **angles arrondis**, une hiérarchie visuelle claire et de **légères ombres** afin de donner de la profondeur aux différents éléments sans alourdir l'interface.

L'ensemble des maquettes réalisées est disponible en annexe.



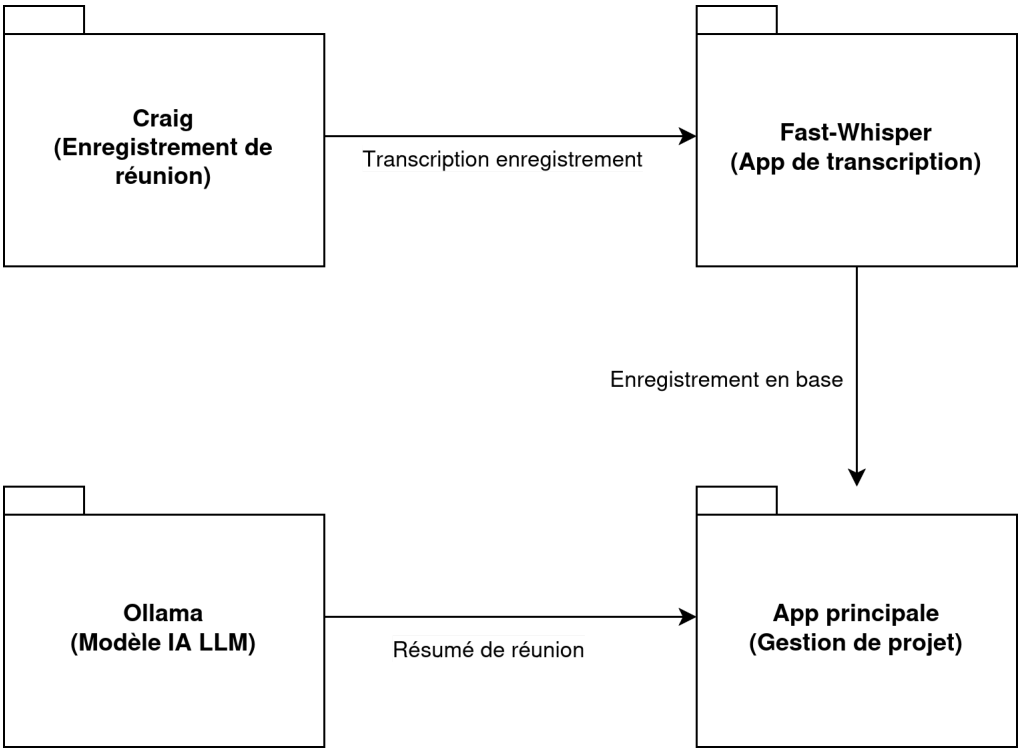


Light mode

Dark mode

4. Architecture technique

4.1 Vue d'ensemble



4.2 Environnement de développement

Pour ce projet, j'ai travaillé sur un ordinateur sous Ubuntu. Ce choix m'a permis de disposer d'un environnement adapté au développement web et de faciliter l'installation et l'utilisation des différents outils nécessaires au projet.

J'ai utilisé **PhpStorm** comme environnement de développement (IDE), notamment pour son intégration avec les technologies utilisées dans le projet, telles que **Symfony, PHP, JavaScript, HTML et CSS**.

Pour le **versionnement du code source**, j'ai utilisé **Git**, avec **GitHub** comme plateforme de gestion du dépôt. Cela m'a permis de conserver un historique des modifications et de sauvegarder le code du projet sur un dépôt distant.

J'ai également expérimenté avec le système de **branches Git** lors du développement de certaines fonctionnalités. Même si mon utilisation de Git n'a pas toujours été aussi structurée que dans un projet professionnel en équipe, cette expérience m'a permis de mieux comprendre l'intérêt du versionnement et de l'isolation des développements.

GitHub m'a également permis d'utiliser les **Issues** et les **Milestones** pour organiser différentes tâches et étapes du projet. Leur utilisation n'a cependant pas été systématique tout au long du développement. Avec le recul, je mettrais davantage en place cette organisation dès le début du projet afin d'avoir un meilleur suivi de l'avancement.

Enfin, l'utilisation de Git et de GitHub constitue une base pour mettre en place par la suite une démarche d'**intégration et de déploiement continus (CI/CD)**, notamment afin d'automatiser les tests, les vérifications du code et le déploiement de l'application.

J'ai aussi utilisé **Docker** et **Docker Compose** pour conteneuriser les différents services nécessaires au fonctionnement de l'application. Chaque composant dispose ainsi de son propre environnement, avec ses dépendances et sa configuration.

L'utilisation de fichiers de configuration me permet de définir les paramètres de chaque service, notamment les dépendances, les variables d'environnement et les interactions entre les conteneurs. Cette approche facilite la modification de l'environnement et l'ajout de nouveaux services sans avoir à reconfigurer manuellement l'ensemble de la machine.

Le projet étant composé de plusieurs ensembles de services Docker Compose, j'ai également créé un **Makefile** afin de simplifier leur utilisation. Celui-ci regroupe les commandes courantes permettant notamment de démarrer, d'arrêter ou de reconstruire les environnements à l'aide de commandes simples.

Cette organisation me permet de gagner du temps lors du développement, de centraliser les opérations courantes et de faciliter la maintenance de l'environnement technique. Elle constitue également une base pour reproduire plus facilement cet environnement sur une autre machine.

4.3 Technologies utilisées

4.3.1 Application principale

Pour la partie DWWM, j'ai choisi de développer une **application monolithique avec Symfony**, en conservant la possibilité de faire évoluer l'architecture par la suite, notamment en développant un ou plusieurs frontends avec Vue.js.

Ce choix me permet de concentrer le développement sur une application complète et fonctionnelle, tout en utilisant les compétences acquises durant ma formation. L'application suit l'architecture **MVC (Modèle-Vue-Contrôleur)** : Symfony gère la

logique côté serveur, tandis que Twig permet de générer les pages HTML affichées dans le navigateur.

Le langage utilisé côté serveur est **PHP**, en version 8.5. Les pages sont générées avec **Twig**, un moteur de templates qui facilite la séparation entre la logique de l'application et la présentation des informations.

Symfony fournit également plusieurs composants qui facilitent le développement et permettent de structurer l'application.

Doctrine, un ORM (*Object-Relational Mapper*) qui fait le lien entre les objets PHP et les tables de la base de données. Il permet notamment de représenter les données sous forme d'entités et de manipuler ces données sans devoir écrire systématiquement des requêtes SQL.

Grâce à Doctrine et à son système de migrations, j'ai pu définir la structure de mes données dans le code, puis appliquer les changements nécessaires à la base de données à l'aide de commandes.

J'utilise également **API Platform** pour exposer certaines fonctionnalités de l'application sous forme d'API. Cela permet notamment d'effectuer des échanges de données asynchrones depuis l'interface utilisateur, sans avoir à recharger entièrement la page.

Cette approche est utilisée pour certaines interactions dynamiques de l'application. Elle pourra également faciliter une éventuelle évolution vers un frontend séparé, même si l'application actuelle repose principalement sur Symfony et Twig.

Un autre composant est **Mercure,** qui permet de transmettre des mises à jour en temps réel aux navigateurs qui se sont abonnés aux flux correspondants. Il évite ainsi d'avoir à interroger régulièrement le serveur pour savoir si une information a changé.

Dans mon application, je l'utilise pour actualiser l'affichage de la transcription d'une réunion lorsqu'elle est terminée. À terme, cette technologie pourrait également servir à développer d'autres fonctionnalités en temps réel, comme un système de messagerie entre utilisateurs.

Le hub Mercure fonctionne dans son propre conteneur Docker. Les navigateurs peuvent s'y abonner pour recevoir les mises à jour publiées par l'application. Caddy intervient dans l'accès aux services selon la configuration du serveur.

J'ai aussi mis en place **Symfony Messenger** permet de gérer des traitements asynchrones en plaçant des messages dans un transport, puis en les faisant traiter par un worker.

Je l’utilise pour gérer la détection automatique de nouveaux enregistrements de réunion et leur ajout en base de données. Ce mécanisme permet de séparer certains traitements du déroulement classique des requêtes HTTP.

Le traitement des messages est exécuté par un worker dans un conteneur dédié.

Pour pouvoir traduire mon application, j’utilise aussi **Translation** qui est un module qui permet de détecter la locale facilement de manière à changer le texte en fonction de la langue de l’utilisateur. Je l’ai intégré de manière à avoir des textes en français et en anglais et permettre l’ajout d’autres langues dans le futur.

Gestion des ressources front-end

Pour préparer les ressources nécessaires à l’interface, j’utilise **Webpack Encore**, qui s’appuie sur Webpack et Node.js.

Il permet de regrouper et de compiler les fichiers JavaScript et CSS, de gérer leurs dépendances et de produire les ressources optimisées utilisées par les pages. Il peut également générer plusieurs fichiers distincts afin de ne charger que les ressources nécessaires.

Webpack Encore est exécuté dans un environnement Node.js, isolé dans un conteneur Docker. Cela permet de séparer les outils de compilation du reste de l’application.

Base de données

L’application utilise **MariaDB**, un système de gestion de base de données relationnelle libre, issu de MySQL. Il permet de stocker les données de l’application dans des tables liées entre elles par des relations.

Cette organisation est adaptée aux données de mon projet, notamment les utilisateurs, les projets, les tâches et les réunions. Les relations entre les tables permettent de structurer ces informations et de garantir la cohérence des données.

MariaDB fonctionne dans son propre conteneur Docker, indépendamment de l’application Symfony.

Pour faciliter l’administration de la base de données, j’utilise également **phpMyAdmin**, une interface graphique permettant notamment de consulter les tables, d’exécuter des requêtes SQL et d'administrer les données. Son accès est limité à l’environnement prévu pour son administration et n’est pas exposé publiquement.

Représentation de la base de données en annexe

Serveur web

Enfin, j'utilise **Caddy** comme serveur web et point d'entrée vers l'application. Il reçoit les requêtes et les dirige vers les services correspondants, notamment l'application Symfony et, selon la configuration, le hub Mercure.

Caddy a l'avantage de disposer d'une configuration relativement simple, définie dans un fichier CaddyFile. Il peut également gérer automatiquement les certificats HTTPS.

Il fonctionne dans son propre conteneur Docker, ce qui permet de séparer le rôle de serveur web de celui de l'application PHP.

4.3.2 Enregistrement des réunions

Pour gérer l'enregistrement des réunions, j'ai choisi de m'appuyer sur **Discord**, un outil que je connais et qui est utilisé pendant ma formation.

J'ai retenu **Craig**, un outil open source permettant d'enregistrer les conversations vocales sur Discord. L'un de ses principaux avantages est la possibilité d'effectuer des enregistrements multipistes, c'est-à-dire de disposer d'une piste audio distincte pour chaque participant. Cette séparation facilite l'identification des intervenants lors de la transcription et permet de conserver les informations sur l'origine des différentes prises de parole.

Le caractère open source de Craig m'a également permis de l'héberger dans mon propre environnement et de modifier son fonctionnement selon les besoins du projet. J'ai notamment adapté le service afin d'ajouter un endpoint API permettant à mon application d'accéder aux enregistrements effectués.

Craig fonctionne dans un environnement Docker Compose distinct de celui de l'application principale. Cet environnement comprend trois conteneurs : l'application Craig, une base de données **PostgreSQL** pour le stockage des données et **Redis**, utilisé pour certaines opérations nécessitant un accès rapide aux données.

Cette séparation permet d'isoler le service d'enregistrement du reste de l'application et de faciliter sa configuration et sa maintenance.

4.3.3 Transcription audio

J'ai choisi de séparer le processus de transcription de l'application principale, car celui-ci nécessite l'exécution de traitements spécifiques en Python et peut mobiliser des ressources importantes, notamment lors de l'utilisation du processeur graphique (GPU).

Pour cela, j'ai développé un service intermédiaire en **Node.js**, utilisant **Express.js** pour exposer des endpoints HTTP. Ce service communique avec les autres composants de l'environnement Docker et coordonne les différentes étapes du traitement des enregistrements.

Lorsqu'une transcription doit être réalisée, le service récupère l'enregistrement auprès de Craig, prépare les pistes audio, puis lance le programme Python chargé de la transcription.

Ce programme utilise **Faster-Whisper**, une implémentation optimisée du modèle Whisper, qui permet de convertir les données audio en texte. Chaque piste est traitée séparément afin de conserver les informations permettant d'identifier les différents intervenants. Les segments de transcription sont ensuite organisés chronologiquement et peuvent être regroupés lorsque plusieurs segments consécutifs d'un même participant correspondent à une prise de parole continue.

Une fois le traitement terminé, le service transmet les résultats à l'application Symfony, qui peut alors les enregistrer et les exploiter.

Le service Node.js est isolé dans son propre conteneur. Ses endpoints sont accessibles uniquement depuis les autres conteneurs autorisés sur le réseau Docker prévu à cet effet, et ne sont pas directement exposés à l'extérieur.

Cette architecture permet de séparer la gestion de l'application, l'orchestration du traitement audio et le calcul de la transcription. Elle facilite également l'évolution du système, puisque le service de transcription peut être modifié indépendamment de l'application principale.

4.3.4 Intelligence Artificielle locale et traitement des transcriptions

Pour compléter le système de transcription, j'ai intégré **Ollama**, un outil permettant d'exécuter des modèles de langage (LLM, Large Language Models) localement et de les interroger à travers une API.

Contrairement à un service d'IA hébergé par un fournisseur externe, cette solution me permet de faire fonctionner les modèles dans mon propre environnement. Ce choix répond notamment à un objectif de maîtrise des coûts et de confidentialité : les données transmises au modèle peuvent être traitées localement, sans devoir être envoyées à une API d'IA externe.

Compte tenu des ressources matérielles disponibles, j'utilise actuellement le modèle **Qwen3 8B**. Ce choix constitue un compromis entre les capacités du modèle et les ressources nécessaires à son exécution. Ollama permet toutefois de remplacer ce modèle par un autre, en fonction des performances recherchées et des capacités du serveur.

Dans le cadre du projet, cette brique d'intelligence artificielle a vocation à exploiter les transcriptions afin d'en extraire des informations utiles à la gestion de projet, notamment l'identification des actions évoquées pendant les réunions et la proposition de tâches correspondantes.

L'objectif est de rapprocher les échanges effectués pendant les réunions du suivi opérationnel du projet. Cette fonctionnalité constitue un complément à la gestion classique des tâches et offre des possibilités d'évolution pour la suite du projet.

5. Développement Back-end

5.1 Base de donnée

Pour gérer les données de l'application, j'ai utilisé **Doctrine ORM**, qui permet de faire le lien entre les objets PHP et les tables de la base de données relationnelle.

J'ai commencé par créer les entités correspondant aux éléments définis dans mon diagramme de classes, à l'aide de la commande **symfony console make:entity**. Une entité représente un objet métier de l'application et contient les propriétés qui permettent de décrire ses données. Grâce au mapping Doctrine, ces propriétés sont associées aux colonnes d'une table en base de données.

Cette commande permet également de créer un repository associé à l'entité. Celui-ci regroupe les méthodes permettant de récupérer les données correspondantes. Doctrine fournit notamment des méthodes courantes comme **find(), findAll(), findBy() et findOneBy()**. Il est également possible d'implémenter des méthodes personnalisées, notamment à l'aide du QueryBuilder, afin de construire des requêtes adaptées aux besoins de l'application.

Pour la gestion des utilisateurs, j'ai utilisé la commande **symfony console make:user.** Elle permet de générer une classe utilisateur compatible avec le système de sécurité de Symfony. La gestion des mots de passe peut ainsi s'appuyer sur le mécanisme de hachage prévu par le framework.

Une fois les entités créées et leurs relations définies, j'ai utilisé les migrations Doctrine pour synchroniser leur structure avec celle de la base de données. La commande **symfony console make:migration** génère un fichier de commandes SQL décrivant les modifications nécessaires. Après avoir vérifié les changements proposés, je les applique avec la commande **symfony console doctrine:migrations:migrate**. Cette méthode permet de conserver un historique des évolutions du schéma de la base de données.

Pour illustrer l'utilisation des repositories, voici un exemple de méthode développée pour l'entité **Task**. Elle permet de récupérer les tâches d'un projet en joignant leur contenu actuel ainsi que les informations relatives à l'utilisateur auquel elles sont attribuées.

```php
public function findTasksByProjectId(string $projectId): array
{
    return $this->createQueryBuilder( alias: 't')
        ->select( ...select: 't', 'tc', 'pu', 'u')
        ->join( join: 't.currentContent', alias: 'tc')
        ->join( join: 't.project', alias: 'p')
        ->leftJoin( join: 'tc.attributedTo', alias: 'pu')
        ->leftJoin( join: 'pu.user', alias: 'u')
        ->where( ...predicates: 'p.id = :projectId')
        ->setParameter( key: 'projectId', $projectId)
        ->orderBy( sort: 'tc.status', order: 'DESC')
        ->getQuery()
        ->getResult();
}
```

Méthode findTasksByProjectId()

```sql
SELECT t.*, tc.*, pu.*, u.*
FROM task AS t
JOIN task_content AS tc
    ON t.current_content_id = tc.id
JOIN project AS p
    ON t.project_id = p.id
LEFT JOIN project_user AS pu
    ON tc.attributed_to_id = pu.id
LEFT JOIN user AS u
    ON pu.user_id = u.id
WHERE p.id = :projectId
ORDER BY tc.status DESC;
```

Equivalent SQL

5.2 Contrôleurs

Les contrôleurs constituent la couche Controller de l'architecture MVC (Modèle-Vue-Contrôleur). Ils font le lien entre les requêtes de l'utilisateur, les traitements de l'application et les vues à afficher. Dans Symfony, je peux créer un contrôleur à l'aide de la commande **symfony console make:controller** ainsi que des contrôleurs spécifiques à l'inscription et la connexion avec respectivement **symfony console make:security:form-login** et **symfony console make:registration-form.**

Chaque méthode d'un contrôleur peut être associée à une ou plusieurs routes. Ces routes permettent à Symfony de déterminer quelle méthode exécuter en fonction de l'URL demandée. J'utilise l'attribut #[Route] pour définir les routes de mon application.

J'ai notamment configuré certaines routes pour prendre en charge plusieurs langues. Cela permet d'avoir des URL adaptées au français et à l'anglais tout en conservant un nom de route commun, utilisable dans le reste de l'application.

Par exemple, dans mon contrôleur de projets, je peux définir une route par défaut permettant d'accéder aux routes en lien avec un projet :

```php
#[Route(path: ['fr' => 'projet', 'project'], name: 'app_project_')]
final class ProjectController extends AbstractController
```

Puis je peux ajouter une route spécifique grâce à une méthode pour accéder aux tâches de ce projet en utilisant un autre attribut #[Route] qui s'ajoute à la précédente :

```
#[Route(path: ['fr' => '/{id:project}/taches', '/{id:project}/tasks'], name:
'tasks')]
public function tasks(Project $project, UserRepository $userRepository,
TaskRepository $taskRepository): Response
{
   $loggedUser = $this->getUser();
   if ($loggedUser === null) {
       return $this->redirectToRoute('app_login');
   }

   $user = $userRepository->findOneBy(['username' =>
$loggedUser->getUserIdentifier()]);

   if (!$this->isUserInProject($project, $user)) {
      return $this->redirectToRoute('app_home');
   }

   $tasks = $taskRepository->findTasksByProjectId($project->getId());

   return $this->render('project/tasks.html.twig', [
       'user' => $user,
       'tasks' => $tasks,
       'project' => $project,
       'statuses' => TaskContent::STATUSES,
   ]);
}
```

Dans cet exemple, l'attribut #[Route] définit les URL permettant d'accéder à la page des tâches d'un projet, en français ou en anglais. Le paramètre name permet de référencer cette route dans l'application, notamment pour générer des liens.

Le paramètre Project $project illustre également le mécanisme de résolution automatique des entités de Symfony. Grâce à la syntaxe id:project, Symfony utilise l'identifiant présent dans l'URL pour retrouver l'entité Project correspondante et la transmettre à la méthode du contrôleur.

La méthode commence ensuite par vérifier si un utilisateur est connecté. Si aucun utilisateur n'est authentifié, il est redirigé vers la page de connexion. Dans le cas contraire, le contrôleur récupère l'entité User correspondante à partir du dépôt UserRepository.

Il vérifie ensuite si l'utilisateur a bien le droit d'accéder au projet, le cas échéant, il est redirigé vers la page principale.

Il utilise ensuite TaskRepository pour récupérer les tâches associées au projet. Enfin, la méthode appelle render() afin d'afficher le template Twig project/tasks.html.twig. Elle lui transmet les données nécessaires à la construction de la page : l'utilisateur, le projet, les tâches et les différents statuts disponibles.

5.3 Services

En complément des contrôleurs et des repositories, j'ai créé plusieurs **services** afin de regrouper certaines responsabilités et de **séparer la logique métier** des contrôleurs.

Les contrôleurs ont principalement pour rôle de recevoir les requêtes des utilisateurs, de faire appel aux composants nécessaires au traitement de ces requêtes, puis de retourner une réponse. Les services permettent de centraliser des traitements spécifiques afin de pouvoir les réutiliser à différents endroits de l'application.

Dans mon projet, certains services sont notamment dédiés à la communication avec les applications externes utilisées pour le traitement des réunions et des transcriptions. Ils permettent de faire le lien entre Symfony et les différents composants de cette architecture, notamment le service de transcription, Craig pour les enregistrements audio et Ollama pour l'intelligence artificielle locale.

Cette séparation permet de ne pas regrouper toute la logique de communication avec ces applications dans les contrôleurs. Elle facilite également la maintenance du code : si le fonctionnement d'une communication externe évolue, les modifications peuvent être concentrées dans le service concerné, plutôt que réparties dans plusieurs contrôleurs.

Pour illustrer cette organisation, je présente ci-dessous un exemple de service issu de mon application.

```php
public function transcribe(Meeting $meeting): void
{
    if ($meeting->getTranscript()) {
        return;
    }

    $meeting->setStatus(Status::PROCESSING);
    $this->entityManager->flush();

    try {
        $response = $this->httpClient->request(
            'POST',
            $this->transcriberUrl . '/transcribe',
            [
                'headers' => [
                    'Content-Type' => 'application/json',
                ],
                'json' => [
                    'id' => $meeting->getCraigId(),
```

```
                    'key' => $meeting->getCraigKey(),
                ],
            ]
        );
    } catch (TransportExceptionInterface $e) {
        $meeting->setStatus(Status::PENDING);
        $this->entityManager->flush();
        throw $e;
    }

    if ($response->getStatusCode() >= 400) {
        throw new \RuntimeException(
            'Transcription service returned HTTP ' .
            $response->getStatusCode()
        );
    }
}
```

La méthode **transcribe()** reçoit une entité Meeting en paramètre. Elle vérifie d'abord si une transcription existe déjà. Si c'est le cas, elle arrête le traitement afin d'éviter de lancer inutilement une nouvelle demande.

Dans le cas contraire, elle modifie le statut de la réunion pour indiquer que son traitement est en cours, puis enregistre cette modification en base de données grâce à Doctrine.

Le service utilise ensuite le composant HTTP Client de Symfony pour envoyer une requête POST au service de transcription. Cette requête contient l'identifiant et la clé de l'enregistrement nécessaires pour retrouver le fichier audio à traiter.

J'ai également prévu une gestion des erreurs de communication. Si une exception de transport est levée lors de la requête, le statut de la réunion est réinitialisé à PENDING, la modification est enregistrée, puis l'exception est propagée. Le service vérifie aussi le code HTTP retourné et lève une exception si celui-ci indique une erreur.

5.4 API

À l'aide d'API Platform, j'ai pu mettre en place une API sur certaines entités de mon application en utilisant l'attribut #[ApiResource]. Cette configuration permet de définir les opérations disponibles sur une ressource, comme la récupération de données (GET), la création (POST) ou la modification (PATCH et PUT). Il est également possible de créer des opérations personnalisées pour répondre à des besoins spécifiques de l'application.

Par exemple, je souhaitais pouvoir modifier le statut d'une tâche en la déplaçant d'une colonne à une autre dans l'interface, sans avoir à recharger la page. J'ai donc créé une opération POST personnalisée sur l'entité Task, accessible à l'URI tasks/{id}/revision.

```
#[ApiResource(
   operations: [
       new Post(
           uriTemplate: 'tasks/{id}/revision',
           input: UpdateTaskDto::class,
           processor: CreateTaskRevisionProcessor::class,
       ),
   ]
)]
```

Le paramètre input indique à API Platform d'utiliser un **Data Transfer Object (DTO),** UpdateTaskDto, comme structure des données reçues. Cela me permet de contrôler les champs modifiables par cette opération, sans exposer directement l'ensemble de l'entité. Le paramètre processor désigne la classe chargée de traiter ces données après leur réception.

J'ai créé la classe CreateTaskRevisionProcessor pour gérer le traitement de la requête envoyée à mon API. Avant de modifier une tâche, le Processor effectue plusieurs vérifications afin de s'assurer que l'utilisateur dispose des droits nécessaires.

Tout d'abord, il récupère l'utilisateur connecté grâce au composant de sécurité de Symfony. Il recherche ensuite la tâche concernée à partir de son identifiant, transmis dans l'URL. Il vérifie que l'utilisateur appartient bien au projet auquel cette tâche est rattachée.

Enfin, il contrôle les droits de modification de la tâche. L'utilisateur doit être la personne à qui la tâche est attribuée, son créateur ou disposer de la permission d'administration du projet.

Si ces vérifications sont satisfaites, le Processor appelle TaskService pour créer une nouvelle version du contenu de la tâche. Cette version est ensuite enregistrée en base de données à l'aide de Doctrine.

Cette organisation permet de séparer les responsabilités : le Processor vérifie les conditions d'accès et coordonne le traitement de la requête, tandis que le service métier gère la création de la nouvelle version de la tâche.

```
public function process(mixed $data, Operation $operation, array $uriVariables =
[], array $context = []): void
```

```
{
    $user = $this->security->getUser();

    if (!$user) {
        throw new AccessDeniedException('No authenticated user.');
    }

    $task = $this->taskRepository->find($uriVariables['id']);

    if (!$task) {
        throw new PropertyNotFoundException('Task not found');
    }

    $projectUser = $this->projectUserRepository->findOneBy([
        'user' => $user,
        'project' => $task->getProject(),
    ]);

    if (!$projectUser) {
        throw new AccessDeniedException('User not in project');
    }

    if ($task->getCurrentContent()->getAttributedTo() !== $projectUser &&
$task->getCreatedBy() !== $projectUser && !in_array("ROLE_ADMIN",
$projectUser->getPermissions()))  {
        throw new AccessDeniedException('User not allowed to modify task');
    }

    $revision = $this->service->createRevision($task, $data, $projectUser);

    $this->em->persist($revision);
    $this->em->flush();
}
```

J'ai également développé d'autres endpoints personnalisés. Le premier permet au service de transcription d'envoyer une requête POST à l'application lorsque la transcription d'un enregistrement est terminée. Cela permet à l'application de récupérer ou de traiter les informations liées à la transcription.

Un autre endpoint permet d'ajouter un utilisateur à un projet sans recharger la page, ce qui rend l'interface plus fluide.

À l'avenir, j'aimerais étendre l'utilisation de l'API à davantage de modifications dans l'application. L'objectif serait de pouvoir modifier les tâches directement depuis la page du projet, sans rechargement complet, afin d'améliorer l'expérience utilisateur et de rendre l'interface plus réactive.

6. Développement Front-end

6.1 Structure des interfaces

Pour développer les interfaces de mon application, j'utilise **Twig**, le moteur de templates intégré à Symfony. Il me permet d'afficher les données transmises par les contrôleurs et de structurer le contenu des pages grâce à des instructions spécifiques.

Twig facilite également la réutilisation des éléments d'interface grâce à un système d'héritage et d'inclusion de templates. Par exemple, un template principal base.html.twig définit la structure commune des pages. Il contient des blocs que les autres templates peuvent compléter ou remplacer pour afficher leur propre contenu.

Dans mon application, le fichier base.html.twig gère notamment l'intégration des fichiers JavaScript et CSS, l'affichage de la barre de navigation et, lorsque l'utilisateur est connecté, celui de la barre latérale. Il définit également un bloc body, dans lequel chaque page insère son contenu spécifique.

Chaque page possède ensuite son propre template Twig, qui hérite de base.html.twig. J'utilise aussi des **partials**, c'est-à-dire des templates partiels destinés à être inclus dans plusieurs pages. Ils me permettent de réutiliser certains composants de l'interface, comme les cartes de tâches ou les onglets des projets présents dans la barre latérale. Cette organisation évite de dupliquer du code et facilite la maintenance de l'application.

Les fichiers JavaScript, CSS et les médias sont regroupés dans le répertoire assets. J'utilise **Webpack Encore** pour gérer la compilation et la préparation des ressources front-end. Sa configuration se trouve dans le fichier webpack.config.js, dans lequel je définis notamment les points d'entrée correspondant aux fichiers à compiler.

Une fois les ressources compilées, elles sont générées dans le répertoire `public`, afin de pouvoir être utilisées par le navigateur. Dans les templates Twig, j'intègre les fichiers générés à l'aide des fonctions fournies par Webpack Encore. Par exemple :

```
{{ encore_entry_script_tags('scripts') }}
{{ encore_entry_link_tags('styles') }}
```

La première instruction ajoute les fichiers JavaScript associés au point d'entrée scripts, tandis que la seconde ajoute les fichiers CSS associés au point d'entrée styles.

6.2 Intégration graphique

Pour intégrer les maquettes de l'application, j'ai utilisé **Bootstrap**, un framework CSS qui facilite la création d'interfaces responsives, c'est-à-dire capables de s'adapter à différentes tailles d'écran.

Bootstrap propose notamment un système de grille basé sur des colonnes et des breakpoints. Ces derniers permettent de définir des changements de disposition en fonction de la largeur de l'écran. Il est ainsi possible d'adapter l'organisation des éléments selon que l'application est consultée sur un ordinateur, une tablette ou un téléphone.

Le framework fournit également de nombreuses classes CSS prédéfinies pour les besoins courants, comme la gestion des marges, des espacements, des couleurs et de l'alignement. Cela permet de développer plus rapidement les interfaces sans avoir à recréer systématiquement ces comportements.

Bootstrap offre aussi des possibilités de personnalisation. Il est notamment possible de modifier ses variables CSS pour adapter les couleurs, les arrondis, les ombres ou encore les propriétés typographiques. Il est également possible de surcharger les styles existants en définissant ses propres règles CSS.

Pour adapter Bootstrap à la charte graphique de mon application, j'ai crée trois fichiers de personnalisation :

- bs-override.css, pour les modifications générales ;
- bs-override-light.css, pour les couleurs du thème clair ;
- bs-override-dark.css, pour les couleurs du thème sombre.

Cette séparation me permet de conserver des styles communs aux deux thèmes tout en adaptant les couleurs en fonction du mode choisi par l'utilisateur.

Par exemple, dans le fichier bs-override-light.css, je définis plusieurs variables pour personnaliser les couleurs et l'apparence générale de l'interface

```
[data-bs-theme="light"]:root {
    --bs-primary: #20CC2C;
    --bs-primary-hover: #14b81f;
    --bs-secondary: #FFFFFF;

    --bs-body-bg: #fafcff;
    --bs-body-font-family: Inter;
    --bs-body-color: #323547;

    --shadow-color-sm: rgb(134 153 182 / 0.20);
    --shadow-color: rgb(134 153 182 / 0.35);
```

```
}
```

Ces variables permettent notamment de définir la couleur principale de l'application, la couleur de fond, la police utilisée pour le texte et les couleurs des ombres. Le sélecteur [data-bs-theme="light"] permet de limiter l'application de ces règles aux éléments associés au thème clair.

En complément de ces personnalisations, j'ai créé un fichier default.css pour regrouper mes propres classes CSS. Elles me permettent de définir des styles spécifiques à mon application, qui ne sont pas nécessairement proposés directement par Bootstrap.

Par exemple, j'ai créé la classe avatar-md pour standardiser les dimensions des images de profil :

```
.avatar-md {
   height: 3rem;
   width: 3rem;
   padding: 0;
}
```

Cette classe fixe la hauteur et la largeur des images à 3rem et supprime leur espacement intérieur. Je peux ainsi réutiliser facilement le même style sur les différents éléments de l'application qui affichent un avatar.

J'ai également développé des comportements plus spécifiques en CSS, notamment une animation pour l'affichage du champ de recherche :

```
.search-field {
   transform: translateX(-95%);
   transition: transform .5s;
}

.search-field:hover {
   transform: translateX(-90%);
}

.search-field:focus-within,
.search-field:has(.search-input:not(:placeholder-shown)){
   transform: translateX(0%);
}
```

La classe .search-field positionne initialement le champ de recherche avec un décalage horizontal de -95 %. La propriété transition permet d'animer progressivement les changements de position sur une durée de 0,5 seconde.

Lorsque l'utilisateur survole le champ avec sa souris, grâce au pseudo-sélecteur :hover, celui-ci se déplace légèrement vers la droite, jusqu'à -90 % de sa position initiale.

Enfin, deux conditions permettent d'afficher complètement le champ :

- :focus-within s'applique lorsque le champ ou l'un de ses éléments enfants possède le focus, par exemple lorsque l'utilisateur clique dans le champ de saisie ;
- :has(.search-input:not(:placeholder-shown)) s'applique lorsque le champ .search-input ne présente plus son texte indicatif (placeholder), ce qui permet de maintenir le champ déployé lorsque l'utilisateur a saisi du texte.

Dans ces deux cas, la propriété transform passe à translateX(0%), ce qui ramène le champ à sa position normale.



Barre de recherche cachée



Barre de recherche ouverte

Grâce à ces personnalisations, j'ai pu adapter Bootstrap à la charte graphique de mon application, gérer les thèmes clair et sombre et développer des comportements visuels spécifiques, tout en conservant les avantages du framework.

Captures d'écran de l'intégration en annexe

6.3 JavaScript

J'utilise **JavaScript** pour dynamiser l'interface de mon application en exécutant du code directement dans le navigateur. Cela me permet notamment de réagir aux actions de l'utilisateur, de modifier certains éléments de la page et d'effectuer des requêtes vers mon application sans avoir à recharger systématiquement la page.

J'utilise trois bibliothèques externes installées avec npm :

- **SortableJS**, qui permet de déplacer des éléments par glisser-déposer (*drag and drop*), notamment pour organiser les tâches dans l'interface ;

- **Vis Timeline**, qui facilite la représentation et la manipulation de données sous la forme d'une frise chronologique ;
- **Bootstrap JavaScript**, qui gère les interactions de certains composants de Bootstrap, comme les modales et les menus déroulants.

Pour illustrer l'utilisation de JavaScript dans mon application, je vais présenter le code qui permet d'ajouter un collaborateur à un projet sans recharger la page.

6.3.1 ajout asynchrone d'un collaborateur

```
const addProjectUserModal = Modal.getOrCreateInstance(modalElement);

const addProjectUserFormInput = document.forms["addTeamMemberForm"]["username"];
document.forms["addTeamMemberForm"].addEventListener("submit", addTeamMember);
modalElement.addEventListener('shown.bs.modal', () => {
   addProjectUserFormInput.focus()
})

function addTeamMember(event) {
   event.preventDefault();
   const formInputError =
document.forms["addTeamMemberForm"].querySelector("#usernameToAddFeedback");

   const projectId = addProjectUserFormInput.dataset.projectId;
   const username = addProjectUserFormInput.value;

   if (username === "") {
       return false;
   }

   fetch('/api/project/' + projectId + '/user', {
       method: 'POST',
       headers: {
           'Content-Type': 'application/ld+json',
           'Accept': 'application/ld+json',
           'X-Requested-With': 'XMLHttpRequest',
       },
       body: JSON.stringify({
           username: username,
       }),
   }).then(r => {
       if (!r.ok) {
           addProjectUserFormInput.classList.remove('is-invalid')
           r.json().then(json => { formInputError.innerHTML = json.detail; });
           addProjectUserFormInput.classList.add('is-invalid');
       }
       if (r.ok) {
           addProjectUserFormInput.value = '';
           addProjectUserModal.hide();
```

```
    }
});
```

1. Récupération et initialisation de la modale

Je commence par récupérer l'élément HTML correspondant à la modale d'ajout d'un collaborateur grâce à document.getElementById(). J'utilise ensuite Modal.getOrCreateInstance() pour récupérer l'instance Bootstrap associée à cet élément ou la créer si nécessaire.

Cette instance me permet notamment de contrôler l'affichage de la modale en JavaScript, par exemple pour la fermer après l'ajout d'un collaborateur.

Je récupère ensuite le champ de saisie du nom d'utilisateur à partir du formulaire addTeamMemberForm.

2. Gestion des événements

J'ajoute un event listener avec addEventListener() sur le formulaire. Lorsqu'un événement submit se produit, la fonction addTeamMember() est exécutée.

J'ajoute également un écouteur sur la modale pour détecter l'événement shown.bs.modal, déclenché lorsque celle-ci est entièrement affichée. À ce moment-là, la méthode focus() place automatiquement le curseur dans le champ du nom d'utilisateur. Cela améliore le confort d'utilisation en évitant à l'utilisateur de devoir cliquer lui-même dans le champ.

3. Préparation de la requête

Au début de la fonction addTeamMember(), j'utilise event.preventDefault() pour empêcher l'envoi classique du formulaire, qui entraînerait normalement un rechargement de la page.

Je récupère ensuite l'élément HTML destiné à afficher les messages d'erreur, puis les informations nécessaires à la requête :

- l'identifiant du projet, stocké dans l'attribut data-project-id du champ de saisie
- le nom d'utilisateur saisi par l'utilisateur.

Si le champ est vide, la fonction s'arrête sans envoyer de requête.

4. Envoi de la requête asynchrone

Si le champ contient un nom d'utilisateur, j'utilise la fonction JavaScript fetch() pour envoyer une requête HTTP POST vers l'endpoint /api/project/{projectId}/user.

L’identifiant du projet est intégré à l’URL et le nom d’utilisateur est transmis dans le corps de la requête, au format JSON grâce à JSON.stringify().

L’utilisation de fetch() permet d'effectuer cette communication avec le serveur sans recharger la page. Le serveur peut alors vérifier les données reçues et traiter l’ajout du collaborateur.

5. Traitement de la réponse

Une fois la requête envoyée, j’utilise .then() pour traiter la réponse du serveur. L’objet r représente la réponse HTTP reçue. Je vérifie sa propriété ok, qui indique si le code de statut HTTP est compris entre 200 et 299.

Je distingue alors deux cas :

- En cas d’erreur, si r.ok vaut false, je retire d’abord la classe CSS is-invalid du champ de saisie. Je récupère ensuite le détail du message d’erreur contenu dans la réponse JSON grâce à r.json(), puis je l’affiche dans l’élément prévu à cet effet. Enfin, je réapplique la classe is-invalid afin de signaler visuellement le problème à l’utilisateur.
- En cas de succès, si r.ok vaut true, je vide le champ de saisie et ferme la modale avec addProjectUserModal.hide().

Cette vérification permet de traiter de manière générale les réponses HTTP réussies ou en erreur, sans avoir à tester individuellement chaque code de statut.

7. Déploiement

À ce jour, mon application n’a pas encore été déployée dans un environnement de production. Je vais donc présenter les étapes que j’envisage pour sa mise en ligne, depuis le choix de l’hébergement jusqu’à la configuration des services nécessaires à son fonctionnement.

7.1. Choix de l’hébergement

La première étape consisterait à choisir un environnement d’hébergement adapté aux besoins de l’application.

Une particularité de mon projet est qu’il intègre plusieurs services, dont un service de transcription audio utilisant Faster-Whisper et un modèle d’intelligence artificielle local exécuté avec Ollama. Ces services peuvent nécessiter des ressources matérielles importantes, notamment de la mémoire vive, de l'espace disque et une carte graphique compatible avec CUDA.

Une première possibilité serait d'héberger l'ensemble de l'application sur un **serveur local**, par exemple au sein de l'entreprise. Cette solution permettrait d'éviter les frais de location d'un serveur distant et de disposer d'une carte graphique dédiée aux traitements liés à l'intelligence artificielle.

Une autre possibilité serait d'utiliser un **VPS ou un serveur dédié**. Dans ce cas, il serait envisageable de séparer les services selon leurs besoins :

- un serveur pour l'application web, la base de données et les services associés ;
- un serveur disposant des ressources matérielles nécessaires pour la transcription audio et l'exécution du modèle d'intelligence artificielle.

Cette séparation permettrait d'adapter les ressources à chaque service, mais impliquerait de configurer les communications entre les serveurs et de sécuriser les échanges.

Le choix définitif dépendrait donc du budget disponible, du nombre d'utilisateurs, des besoins en performances et des exigences de disponibilité de l'application.

7.2. Configuration du nom de domaine et du réseau

Une fois l'hébergement choisi, il faudrait configurer un nom de domaine afin de permettre aux utilisateurs d'accéder à l'application à partir d'une adresse facilement mémorisable.

Il faudrait ensuite configurer le DNS pour faire pointer ce nom de domaine vers le serveur hébergeant l'application.

Le serveur web Caddy, déjà utilisé dans mon environnement de développement, pourrait être conservé pour recevoir les requêtes HTTP et HTTPS et les diriger vers les services appropriés.

La configuration réseau devrait également permettre d'exposer uniquement les services nécessaires à l'utilisation de l'application. Les ports accessibles depuis Internet devraient être limités au strict nécessaire. Les services internes, tels que la base de données, phpMyAdmin ou les composants utilisés uniquement par les autres conteneurs, ne devraient pas être directement accessibles depuis l'extérieur.

Enfin, il faudrait configurer le HTTPS afin de chiffrer les communications entre les navigateurs et le serveur. Caddy pourrait faciliter cette étape grâce à sa gestion des certificats TLS.

7.3. Déploiement avec Docker

L'application repose sur Docker et Docker Compose, ce qui devrait faciliter son installation dans un nouvel environnement.

L'objectif serait de pouvoir récupérer le code source depuis le dépôt Git, configurer les variables d'environnement propres à la production, puis construire et démarrer les différents conteneurs à l'aide des fichiers Docker Compose.

Cette méthode permettrait de reproduire l'environnement technique de l'application sans devoir installer manuellement chaque dépendance sur le serveur hôte.

Il faudrait toutefois adapter la configuration à l'environnement de production, notamment pour les paramètres de connexion à la base de données, les URL des services, les secrets de l'application et les paramètres propres à chaque environnement.

8. Difficultés et améliorations

8.1 Difficultés rencontrées

Au cours du développement, j'ai rencontré plusieurs difficultés liées à l'intégration d'applications existantes, parfois peu ou pas documentées. Pour les résoudre, j'ai dû effectuer des tests, analyser le code source et utiliser les outils de débogage disponibles afin de comprendre le fonctionnement des différentes technologies et d'identifier l'origine des problèmes.

8.1.1 Gestion des notifications en temps réel avec Mercure

L'une des difficultés rencontrées concerne l'utilisation de **Mercure**, que j'utilise pour transmettre des notifications au navigateur lorsqu'une entité de l'application est mise à jour.

Lorsque je modifiais une entité depuis l'application principale ou à la suite d'une requête API du client, les notifications étaient correctement reçues. En revanche, lorsqu'une modification était effectuée depuis un autre conteneur Docker, notamment le service de transcription, le navigateur ne recevait pas la notification attendue.

À la fin d'une transcription, ce service envoie une requête à l'API Symfony afin de mettre à jour l'entité Meeting avec le texte transcrit. Je souhaitais que cette modification déclenche une notification Mercure pour permettre à l'interface d'afficher le résultat et d'actualiser son contenu sans rechargement de la page.

Pour comprendre le problème, j'ai utilisé le **profiler Symfony** afin d'examiner le traitement de la requête et de vérifier l'émission des mises à jour. Après avoir corrigé d'autres erreurs, j'ai constaté que Mercure envoyait bien une notification, mais que l'URL associée à la ressource ne correspondait pas à celle à laquelle le navigateur s'était abonné.

En effet, le navigateur accédait à l'application via l'URL https://localhost:8443/api/meeting, tandis que le conteneur de transcription appelait l'API via l'adresse interne http://backend-caddy-1:8000/api/meeting. Ces deux URL désignaient la même application, mais étaient différentes du point de vue de Mercure.

J'ai ainsi compris que l'URL utilisée lors de la requête pouvait influencer l'identifiant de la ressource associée à la notification. Le navigateur étant abonné à une URL différente, il ne recevait pas la mise à jour attendue.

Pour résoudre ce problème, j'ai créé un **listener** chargé d'intercepter l'émission des mises à jour Mercure et de corriger l'URL de la ressource afin qu'elle corresponde à celle utilisée par le client.

8.1.2 Récupération des enregistrements depuis Craig

Une autre difficulté concerne l'accès à la liste des enregistrements de réunion produits par **Craig**, l'application utilisée pour enregistrer les échanges audio.

L'application ne proposait pas, dans la configuration que j'utilisais, de moyen adapté à mon besoin pour récupérer cette liste depuis mon service de transcription.

J'ai donc analysé son fonctionnement et modifié son code pour ajouter un endpoint API permettant d'accéder aux enregistrements disponibles. Cette adaptation m'a ensuite permis d'intégrer leur récupération dans mon processus de transcription.

Cette expérience m'a appris à analyser une application existante et à l'adapter lorsque ses fonctionnalités initiales ne répondent pas entièrement aux besoins de mon projet.

8.1.3 Utilisation de la carte graphique par Docker

Enfin, j'ai rencontré plusieurs difficultés lors de la mise en place du service de transcription automatique des réunions.

Ce service utilise **Faster-Whisper**, une implémentation optimisée du modèle de reconnaissance vocale Whisper. Son exécution peut être accélérée grâce à CUDA, qui permet d'exploiter les capacités de calcul d'une carte graphique NVIDIA.

Cependant, mon application étant exécutée dans un conteneur Docker, celui-ci ne disposait pas initialement de l'accès nécessaire à la carte graphique de mon ordinateur.

J'ai donc dû rechercher la configuration adaptée de Docker Compose, installer et configurer les pilotes et composants nécessaires, puis vérifier que CUDA était correctement accessible depuis le conteneur.

Une fois cet accès fonctionnel, j'ai effectué plusieurs essais pour trouver une configuration de Faster-Whisper adaptée à mon matériel. J'ai choisi le modèle large-v3, qui privilégie la qualité de transcription, et le paramètre compute_type="int8_float16" afin d'utiliser une représentation numérique mixte permettant de réduire les besoins en mémoire tout en exploitant le GPU.

8.2 Améliorations futures

Plusieurs améliorations pourraient être apportées à l'application afin d'enrichir ses fonctionnalités, d'améliorer son architecture et de faciliter sa maintenance.

Approfondir l'intégration de l'intelligence artificielle

La première évolution planifiée est d'ajouter l'analyse des transcriptions de réunions à l'aide d'un modèle d'intelligence artificielle.

L'objectif est de générer automatiquement un résumé des échanges, mais également d'identifier les décisions prises, les actions à réaliser et les tâches évoquées pendant la réunion.

À terme, l'application pourrait proposer la création de nouvelles tâches ou la mise à jour de tâches existantes à partir des informations extraites de la transcription. Ces propositions pourraient être présentées à l'utilisateur, qui pourrait les vérifier et les valider avant leur intégration au projet.

Faire évoluer le front-end vers Vue.js

Actuellement, l'interface de l'application repose principalement sur Symfony et Twig, selon une architecture monolithique dans laquelle le rendu des pages est effectué côté serveur.

Une évolution possible serait de développer le front-end avec **Vue.js** et de faire communiquer cette interface avec le back-end Symfony au moyen d'une API.

Cette séparation permettrait de distinguer plus clairement la gestion des données et de la logique métier, assurées par le back-end, de l'affichage et des interactions avec l'utilisateur, assurés par le front-end.

Elle faciliterait également le développement de nouvelles interfaces, par exemple une application mobile ou une autre interface cliente, qui pourraient réutiliser les mêmes fonctionnalités du back-end.

Cette évolution nécessiterait toutefois de faire évoluer l'API existante et de définir précisément les mécanismes d'authentification et de gestion des droits entre les différentes interfaces.

Mettre en place une chaîne CI/CD

Je souhaiterais également mettre en place une chaîne **CI/CD** (Continuous Integration / Continuous Delivery).

L’objectif serait d’automatiser certaines étapes du cycle de développement, notamment l’exécution des tests et les vérifications du code lors de l’envoi de modifications sur le dépôt Git.

À terme, cette chaîne pourrait également automatiser la construction des images Docker et le déploiement de l’application sur un environnement cible.

Cette amélioration permettrait de détecter plus rapidement certaines régressions, de fiabiliser les mises à jour et de réduire les interventions manuelles lors des déploiements.

Intégrer Microsoft Teams pour l’enregistrement des réunions

Une autre évolution consisterait à intégrer **Microsoft Teams** afin de récupérer les enregistrements des réunions directement depuis cet environnement.

Cette intégration pourrait s’appuyer sur les services Microsoft Azure, avec les autorisations nécessaires pour accéder aux réunions et à leurs enregistrements.

Les fichiers récupérés pourraient ensuite être transmis au service de transcription existant, afin de réutiliser le processus de traitement audio déjà développé.

Cela permettrait d’étendre l’utilisation de l’application aux équipes qui organisent leurs réunions sur Microsoft Teams, sans dépendre uniquement du système d’enregistrement actuellement intégré au projet.

Développer un système de messagerie par projet

Enfin, je souhaiterais ajouter un système de messagerie permettant aux collaborateurs d’échanger directement au sein de chaque projet.

Chaque projet disposerait ainsi de son propre espace de discussion, accessible à ses membres. Les utilisateurs pourraient communiquer sur les tâches en cours, partager des informations et coordonner leurs actions sans avoir à utiliser systématiquement un outil externe.

L’utilisation de Mercure, déjà présent dans l’application, pourrait être étudiée pour transmettre les nouveaux messages en temps réel aux utilisateurs connectés.

Cette fonctionnalité compléterait les outils de gestion de projet existants en centralisant davantage les échanges et les informations liés au travail de l’équipe.

9. Conclusion

La réalisation de ce projet m'a permis de mettre en pratique les compétences acquises durant ma formation de Développeur Web et Web Mobile, en travaillant sur une application concrète répondant à un besoin d'organisation et de suivi de projets en équipe.

J'ai pu développer une application web reposant sur Symfony, Twig et une base de données relationnelle, en mettant en place les principales fonctionnalités de gestion de projets et de tâches. J'ai également approfondi mes connaissances en JavaScript, notamment pour rendre l'interface plus dynamique et permettre des interactions asynchrones avec le serveur. Ce projet m'a ainsi permis de mieux comprendre l'organisation d'une application web, depuis la conception des interfaces et la modélisation des données jusqu'à l'implémentation des fonctionnalités.

Au-delà du développement de l'application principale, j'ai souhaité explorer des technologies plus avancées en intégrant plusieurs services complémentaires, notamment un système d'enregistrement et de transcription de réunions ainsi qu'un modèle d'intelligence artificielle exécuté localement. Ces développements m'ont amené à travailler avec différents langages, outils et environnements, mais aussi à comprendre les problématiques liées à la communication entre plusieurs services, à la configuration de Docker et à l'utilisation des ressources matérielles.

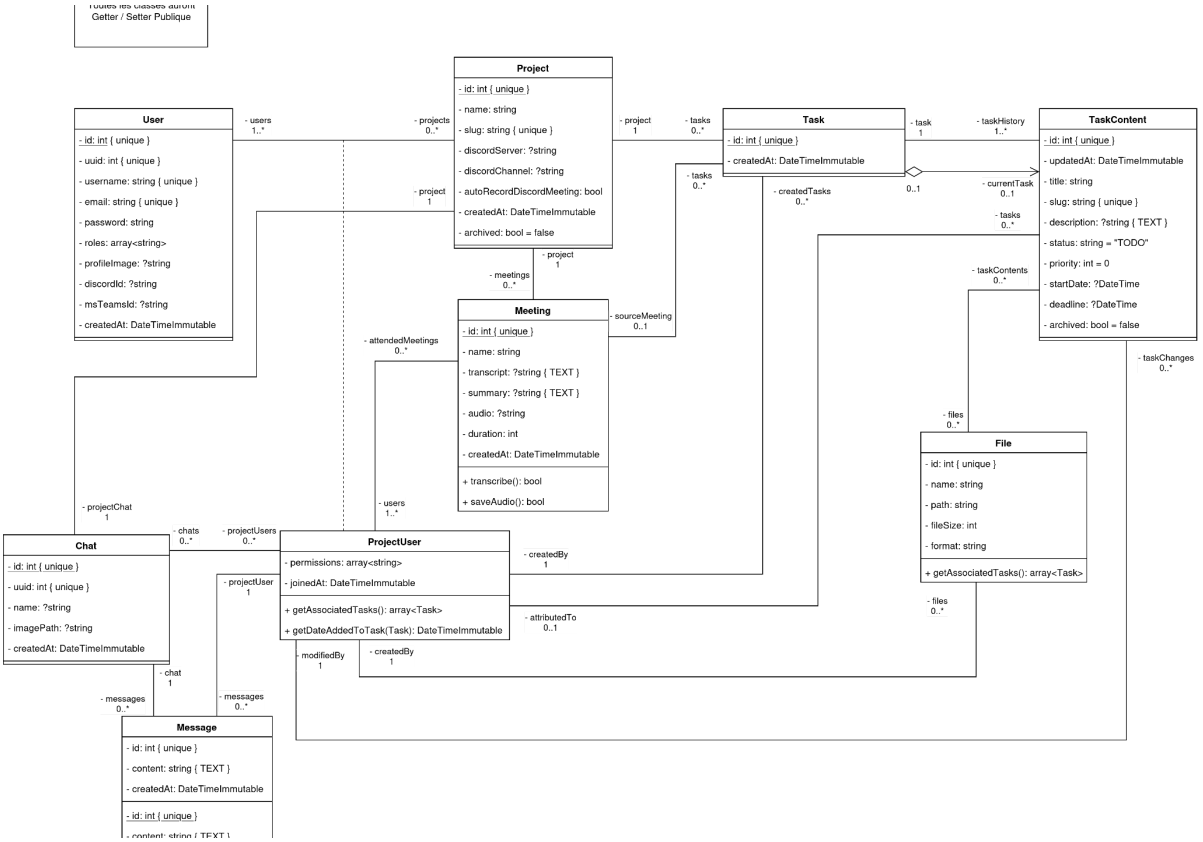
J'ai également rencontré des difficultés techniques qui m'ont obligé à rechercher des solutions, à analyser du code existant et à approfondir le fonctionnement de certaines technologies. Ces expériences m'ont appris à aborder les problèmes de manière méthodique, en m'appuyant sur de la documentation technique et de la lecture de code existant.

Le projet reste perfectible. Plusieurs évolutions sont envisagées, notamment l'intégration plus poussée de l'intelligence artificielle pour exploiter les transcriptions de réunions, le développement d'une messagerie par projet et la mise en place d'une chaîne d'intégration et de déploiement continus. Le déploiement de l'application dans un environnement de production constitue également une étape restant à réaliser.

Enfin, ce projet s'inscrit dans la continuité de mon parcours professionnel. Après l'obtention du titre professionnel Développeur Web et Web Mobile, je souhaite poursuivre mon apprentissage vers le titre de Concepteur Développeur d'Applications. Ce projet m'a permis de consolider mes compétences en développement web tout en découvrant des problématiques plus larges, liées à l'architecture logicielle, à l'intégration de services et à l'intelligence artificielle. Il constitue ainsi une base sur laquelle je pourrai continuer à progresser et à développer des applications plus complètes et plus robustes.

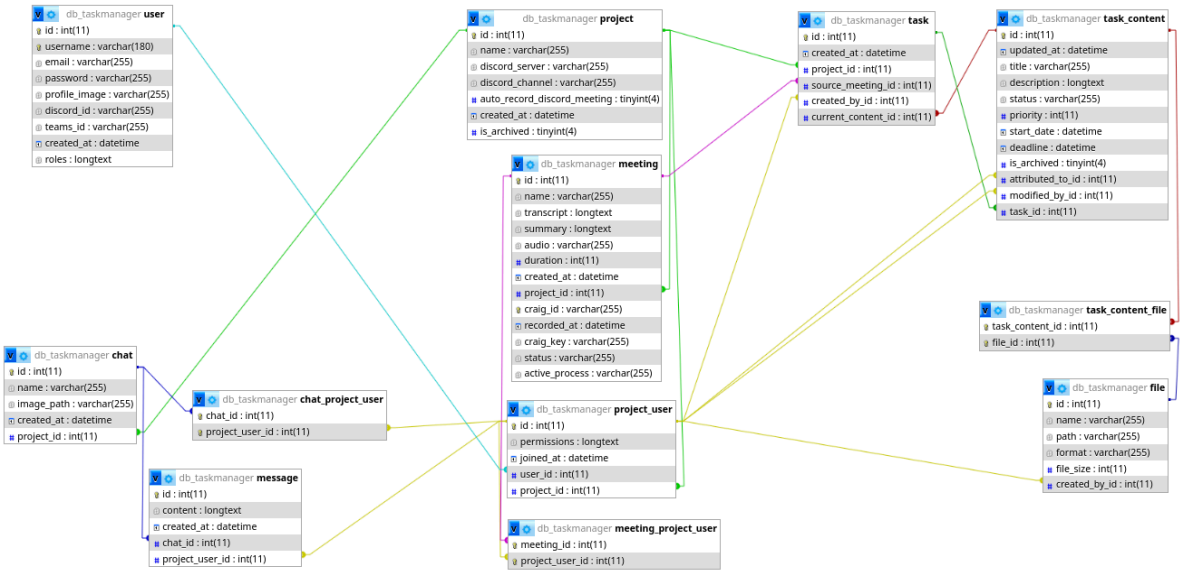
Annexes

Diagramme de classes complet



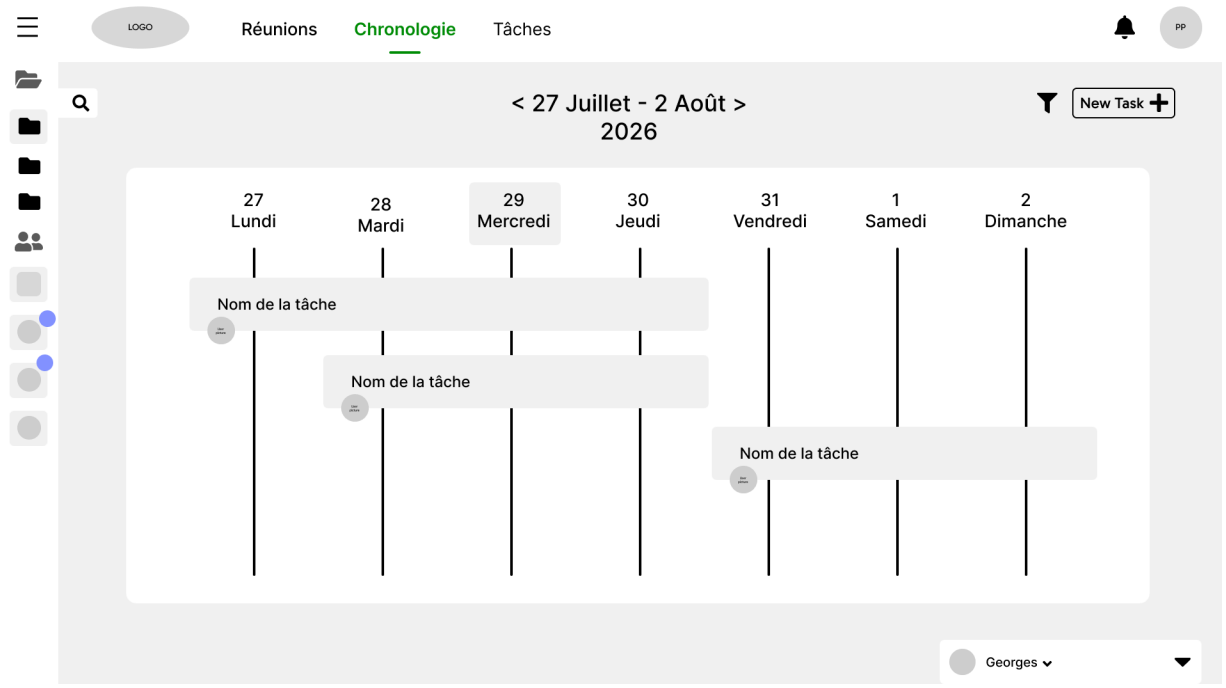
[Revenir en arrière]

Concepteur BDD



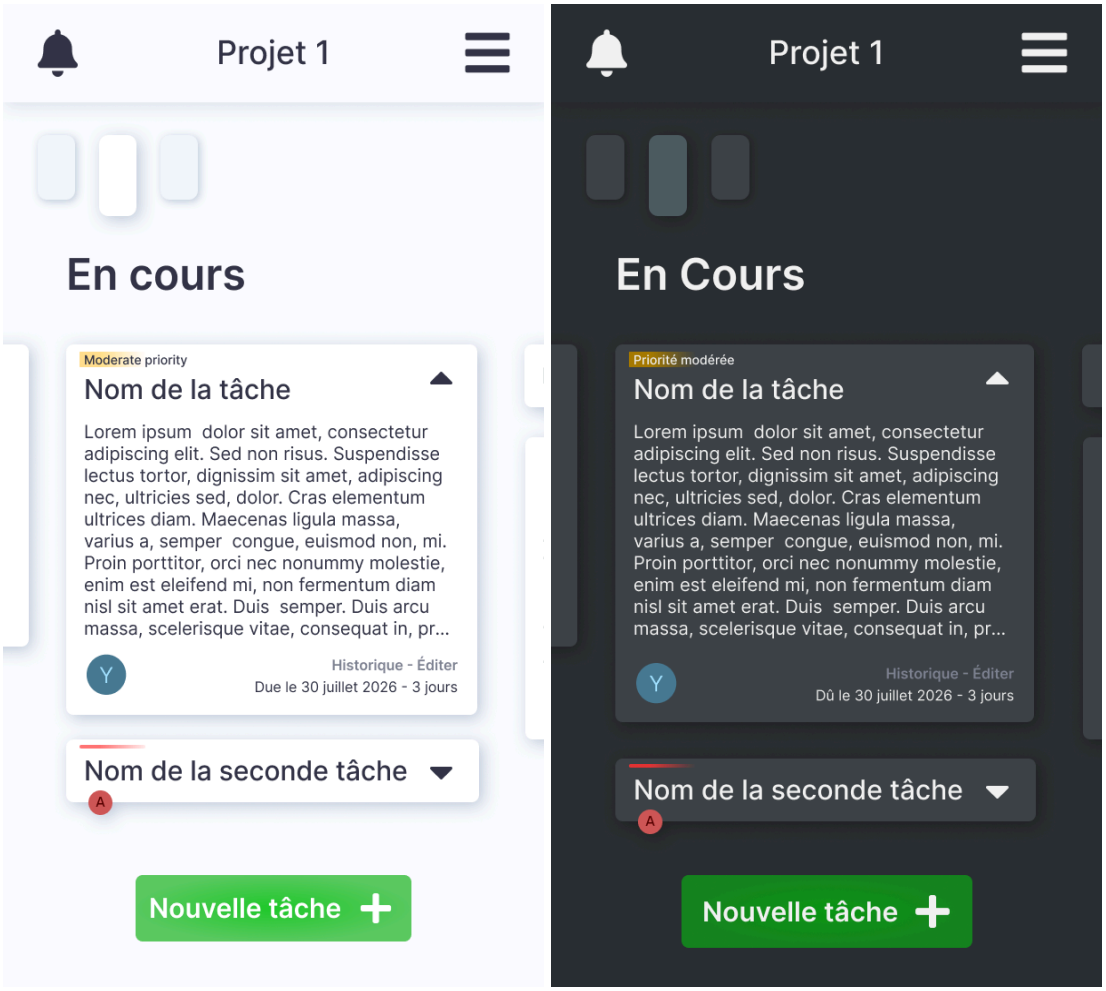
Wireframes

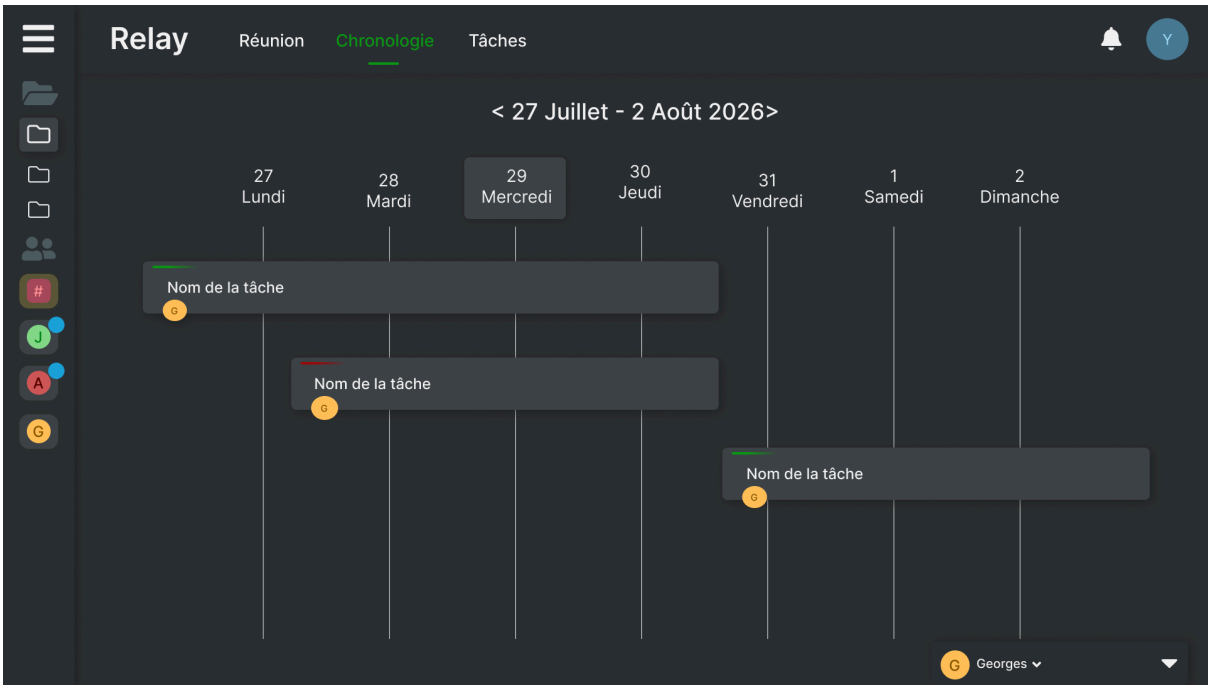


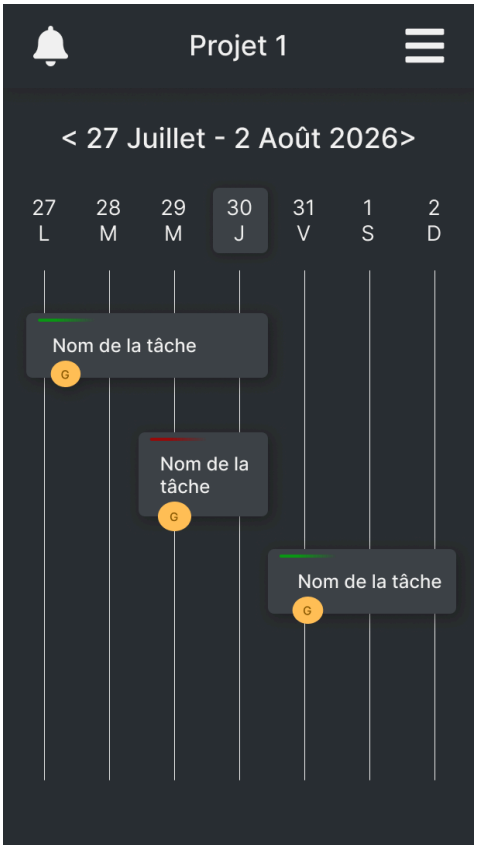


revenir en arrière

Maquettes


Projet 1
En cours
Moderate priority
Nom de la tâche
Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed non risus. Suspendisse lectus tortor, dignissim sit amet, adipiscing nec, ultricies sed, dolor. Cras elementum ultrices diam. Maecenas ligula massa, varius a, semper congue, euismod non, mi. Proin porttitor, orci nec nonummy molestie, enim est eleifend mi, non fermentum diam nisl sit amet erat. Duis semper. Duis arcu massa, scelerisque vitae, consequat in, pr...
Historique - Éditer
Due le 30 juillet 2026 - 3 jours
Nom de la seconde tâche
Nouvelle tâche
Projet 1
En Cours
Priorité modérée
Nom de la tâche
Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed non risus. Suspendisse lectus tortor, dignissim sit amet, adipiscing nec, ultricies sed, dolor. Cras elementum ultrices diam. Maecenas ligula massa, varius a, semper congue, euismod non, mi. Proin porttitor, orci nec nonummy molestie, enim est eleifend mi, non fermentum diam nisl sit amet erat. Duis semper. Duis arcu massa, scelerisque vitae, consequat in, pr...
Historique - Éditer
Dû le 30 juillet 2026 - 3 jours
Nom de la seconde tâche
Nouvelle tâche


Relay
Réunion
Chronologie
Tâches
< 27 Juillet - 2 Août 2026>
27 Lundi
28 Mardi
29 Mercredi
30 Jeudi
31 Vendredi
1 Samedi
2 Dimanche
Nom de la tâche
Nom de la tâche
Nom de la tâche
Georges



revenir en arrière

Task DTO

```
class UpdateTaskDto
{
    public ?string $title = null;
    public ?string $description = null;
    #[Assert\Choice(choices: ['TODO', 'IN_PROGRESS', 'DONE'])]
    public ?string $status = null;
    public ?int $priority = null;
    public ?\DateTimeInterface $startDate = null;
    public ?\DateTimeInterface $deadline = null;
    public ?bool $isArchived = null;
    public ?int $attributedTo = null;
    /**
     * @var int[]|null
     */
    public ?array $fileIds = null;
}
```

revenir en arrière

Captures d’écrans

